ĐẠI HỌC QUỐC GIA SINGAPORE
School of Computing

Exploring New Information Visualization Techniques for VisuAlgo+

Rebuilding Interactive Algorithm Visualizations

(Khám phá các kỹ thuật trực quan hóa thông tin mới cho VisuAlgo+ — xây dựng lại hệ thống trực quan hóa thuật toán tương tác)

Đồ án tốt nghiệp CP4101
Báo cáo đánh giá quá trình (CA)

Truong Van Quoc Bao
A0276275U

Mã dự án (Project ID): H160390
Giảng viên hướng dẫn: A/P Steven Halim
Giám khảo chính (Main Evaluator): SIM Mong Cheng, Terence
Năm học 2026/27, Học kỳ 1
Ngày 4/11/2026

BẢN DỊCH TIẾNG VIỆT ĐẦY ĐỦ

Bản này dùng để đọc và rà soát nội dung. Bản dịch được cập nhật ngày 4/11/2026, đồng bộ với `report/main.tex` trong cùng phiên bản của kho `BJoozz/docsforfyp`.
Giữ nguyên phạm vi, số liệu, mức độ khẳng định và cách phân công trong bản gốc. Tên đề tài đã đăng ký, tên nút trên giao diện, tên thuật toán, định danh mã nguồn và ảnh chụp được giữ lại để thuận tiện đối chiếu. Các tiêu đề, chú thích hình, bảng và sơ đồ được dịch sang tiếng Việt. Không áp dụng giới hạn 15 trang của bản tiếng Anh.

Tóm lược báo cáo

VisuAlgo+ là dự án hợp tác nhằm xây dựng lại VisuAlgo với nền tảng kỹ thuật được đổi mới và giao diện được thiết kế lại, đồng thời giữ nguyên mục tiêu giáo dục. Công việc của tôi tập trung vào lớp trực quan hóa và tương tác: các bộ trực quan hóa đồ thị, cây và sắp xếp; hành vi chỉnh sửa và phát lại của chúng; cùng việc tích hợp vào một giao diện chung. Công việc bắt đầu từ các nguyên mẫu độc lập, sau đó phát triển thành một ứng dụng thống nhất sử dụng bố cục bảng dùng chung.

Báo cáo giải thích việc tách trạng thái đầu vào, bố cục và phát lại; mô hình nhóm bảng V10; các phần mở rộng thuật toán và cách kiểm chứng; cùng các lời giải thích ngắn đặt ngay trên hình vẽ. Ứng dụng hiện tại có 11 thuật toán đồ thị (tính riêng hai biến thể DFS), bảy không gian làm việc cho cây và tám thuật toán sắp xếp. Kết quả kiểm tra được ghi nhận ngày 1/11/2026 gồm 3.273 ca kiểm thử đơn vị và 151 ca kiểm thử trên trình duyệt đều đạt. Những kết quả này cho thấy tiến độ về chức năng. Một đợt thử nghiệm người dùng mang tính định hướng (formative) ở tuần 10 đã dẫn tới vài cải tiến UI/UX, trong đó có floating text trên vùng vẽ và các điều chỉnh cho phần chú giải (legend) và bảng Status. Cả kiểm thử lẫn đợt thử nghiệm đều chưa chứng minh mức cải thiện học tập hay tính dễ sử dụng so với hệ thống khác. Giai đoạn tiếp theo sẽ tăng cường kiểm thử tương tác, đánh giá các nhiệm vụ học tập tiêu biểu và nghiên cứu một phạm vi tích hợp nhỏ với engine WebAssembly riêng biệt.

Cách đọc thuật ngữ. Trong bản dịch, *visualizer* được gọi là bộ trực quan hóa; *canvas* là vùng vẽ; *playback* là phát lại quá trình chạy; *panel* là bảng giao diện; *shell* là khung giao diện chung; *legend* là phần chú giải. Từ *engine* chỉ thành phần thực thi thuật toán. Các nhãn Action, e-Lecture, Pseudocode, Status, Next và Apply giữ nguyên như trên giao diện.

Mục lục

Tóm lược báo cáo **2**

1 Giới thiệu và mục tiêu **3**

2 Cơ sở và các công trình liên quan **4**

3 Quá trình phát triển và phân công **5**

4 Thiết kế và triển khai hệ thống **7**

5 Kết quả hiện tại và kiểm chứng **16**

6 Đánh giá người dùng, giới hạn và bước tiếp theo **17**

7 Kế hoạch cho phần còn lại của dự án **19**

Tài liệu tham khảo **20**

1 Giới thiệu và mục tiêu

1.1 Bối cảnh dự án

VisuAlgo cung cấp các hình thức trực quan hóa tương tác cho cấu trúc dữ liệu và thuật toán. Thiết kế ban đầu đưa nhiều chủ đề vào một môi trường nhất quán và cho phép sinh viên tự khám phá với dữ liệu đầu vào của mình [2]. Những điểm mạnh này là cơ sở xuất phát của VisuAlgo+.

Trong một cuộc trao đổi ban đầu, A/P Steven Halim mô tả hệ công nghệ đang sử dụng, gồm Laravel/PHP, MySQL, D3 và jQuery. Một số công cụ trong đó đã phần nào lỗi thời và không còn phù hợp với các kế hoạch cập nhật trong tương lai, vì vậy mục tiêu của thầy cho phiên bản này là sử dụng những công cụ tốt nhất hiện có cho cả giao diện lẫn engine thực thi. Việc xây dựng lại vẫn giữ những công nghệ hữu ích, trong đó có D3 cho một số tác vụ bố trí và hoạt ảnh.

Mục tiêu là tạo ra một phiên bản kế tiếp hiện đại, giữ được mục đích giáo dục và các tương tác hữu ích của VisuAlgo, đồng thời hỗ trợ việc phát triển sau này. Sản phẩm hiện tại là một ứng dụng thử nghiệm, chưa phải phiên bản thay thế hoàn chỉnh.

1.2 Động lực cá nhân và vai trò

Động lực tham gia VisuAlgo+ của tôi đến từ nền tảng lập trình thi đấu và kinh nghiệm giảng dạy. Việc tham gia nhiều cuộc thi lập trình và đạt kết quả cao đã mang lại cho tôi nhiều trải nghiệm với thuật toán và cấu trúc dữ liệu. Giảng dạy cho những học sinh đam mê lập trình thi đấu giúp tôi giải thích các khái niệm này từ nhiều góc nhìn và nhận ra những điểm người học cần được làm rõ.

Tôi mong muốn vận dụng cả hai trải nghiệm đó vào giao diện và trải nghiệm người dùng của VisuAlgo+, giúp người học theo dõi được vì sao thuật toán đưa ra một quyết định, trạng thái thay đổi ra sao và những thay đổi đó dẫn đến kết quả như thế nào. Góc nhìn này định hướng việc tôi lựa chọn các trạng thái trung gian cần hiển thị, kết nối hình vẽ với mã giả và bổ sung lời giải thích ngắn tại những bước cần làm rõ.

Trách nhiệm ban đầu của tôi là lớp trực quan hóa và tương tác cho thuật toán đồ thị, cấu trúc dữ liệu dạng cây và thuật toán sắp xếp. Công việc bao gồm phát triển các bộ trực quan hóa, hỗ trợ chỉnh sửa đầu vào và phát lại từng bước, rồi tích hợp chúng vào một giao diện nhất quán. Khi phối hợp với Kerk về thiết kế chung và tích hợp, tôi hướng tới việc chuyển kiến thức thuật toán cùng kinh nghiệm giảng dạy của mình thành những cải tiến thực tế, sau đó có thể đánh giá với người học.

1.3 Mục tiêu và phạm vi hiện tại

Dự án có năm mục tiêu làm việc:

1. Lựa chọn các công nghệ hiện đại, phù hợp và được sử dụng rộng rãi để xây dựng lại nền tảng.
2. Giữ lại những chức năng hữu ích của VisuAlgo trong khám phá thuật toán, chỉnh sửa đầu vào và giải thích từng bước.
3. Cung cấp giao diện nhất quán giữa các mô-đun thuật toán và cấu trúc dữ liệu.
4. Xác lập ranh giới trách nhiệm rõ ràng để mở rộng bộ trực quan hóa và tích hợp engine thực thi.
5. Kiểm chứng chức năng đã xây dựng lại, đồng thời xác định những phần còn cần đánh giá kỹ thuật và đánh giá với người dùng.

Kerk Tai Heng là sinh viên FYP cùng giảng viên hướng dẫn với tôi. Kerk phụ trách nền tảng engine WebAssembly (WASM), hệ thống thiết kế website và hạ tầng dùng chung. Mục 3 trình bày rõ hơn về đóng góp và sự phối hợp của chúng tôi.

Ứng dụng hiện tại hướng tới máy tính để bàn và máy tính xách tay. Các thuật toán được cài đặt bằng TypeScript và chạy trong trình duyệt. Engine WebAssembly riêng biệt chưa được nối với các bộ trực quan hóa này. Thiết kế cho điện thoại, mức tương đương chức năng hoàn toàn với VisuAlgo và bằng chứng đo lường về cải thiện học tập nằm ngoài phần công việc đã hoàn thành được trình bày ở đây.

2 Cơ sở và các công trình liên quan

2.1 Trực quan hóa thuật toán trong giáo dục

Halim và cộng sự [2] giới thiệu môi trường thống nhất, tương tác trên web để học thuật toán, về sau trở thành VisuAlgo. Khả năng sử dụng dữ liệu do người học tự nhập và tương tác nhất quán giữa các chủ đề đặc biệt phù hợp với dự án này. Khi xây dựng lại, cần bảo toàn các đặc điểm đó ngay cả khi công nghệ hiển thị và bố cục trang thay đổi. Việc sử dụng lại những ví dụ và thao tác thuật toán quen thuộc cũng tạo ra cơ sở tham chiếu hữu ích khi kiểm tra chức năng được xây dựng lại.

Tuy nhiên, một hoạt ảnh chạy được chưa đủ để chứng minh lợi ích học tập. Nghiên cứu tổng hợp 24 nghiên cứu của Hundhausen, Douglas và Stasko [3] cho thấy cách người học sử dụng trực quan hóa thuật toán quan trọng hơn việc chỉ xét các đặc điểm đồ họa. Điều này dẫn đến việc phân biệt hai câu hỏi trong VisuAlgo+: phần mềm có hoạt động đúng hay không, và sinh viên có thể dùng nó để hiểu, giải thích thuật toán hay không. Các kiểm thử hiện tại chủ yếu trả lời câu hỏi thứ nhất; thử nghiệm tuần 10 (Mục 6.1) và một đợt đánh giá tiếp nối có cấu trúc hướng tới câu hỏi thứ hai.

OpenDSA kết hợp trực quan hóa với nội dung giải thích, bài tập tương tác và đánh giá mức độ thành thạo [4]. Đây là một ví dụ về cách trực quan hóa có thể trở thành một phần trong hoạt động học tập rộng hơn. VisuAlgo+ hiện tập trung vào khám phá: chỉnh sửa đầu vào, đi qua từng bước thực thi, theo dõi mã giả và quan sát trạng

thái. Hệ thống chưa cung cấp một chuỗi hoạt động học tập có đánh giá tương đương. Vì vậy, đóng góp hiện tại nên được xem là nền tảng trực quan hóa, chưa phải một hệ thống giảng dạy hoàn chỉnh.

2.2 Nền tảng tái sử dụng và lựa chọn triển khai

Shaffer và cộng sự [5] khảo sát lĩnh vực trực quan hóa thuật toán và thảo luận các vấn đề như khả năng tiếp cận công cụ, chất lượng và mức độ bao phủ chủ đề. Khảo sát của họ là cơ sở để chú trọng khả năng bảo trì và các mẫu tương tác có thể tái sử dụng.

Về nền tảng kỹ thuật, React Flow cung cấp vùng vẽ tương tác gồm các đỉnh và cạnh [9]. Dự án xây dựng trạng thái theo thuật toán, định danh cạnh và quy tắc trình bày ở lớp phía trên vùng vẽ đó. D3 cung cấp các công cụ tính toán chuyên biệt hơn: mô phỏng lực để xác định vị trí đồ thị và bố trí phân cấp cho cây [1]. Cách kết hợp này tách việc tính vị trí khỏi trách nhiệm quản lý trạng thái ứng dụng. Sắp xếp sử dụng Canvas 2D vì hình thức biểu diễn chính là mảng các cột, thay vì một đồ thị có thể chỉnh sửa.

2.3 Vị trí của công việc này

Dự án áp dụng các ý tưởng đã có về thuật toán, hiển thị và giao diện vào một bài toán xây dựng lại thực tế. Giá trị hiện tại nằm ở việc kết nối chúng một cách nhất quán: dữ liệu có thể chỉnh sửa ổn định, phát lại có thể tái lập, điều khiển trang dùng chung và lời giải thích gắn với từng bước thuật toán. Các bảng có thể ghép/tách và thuật toán low-link không được trình bày như những kỹ thuật nghiên cứu mới.

Ba câu hỏi thiết kế định hướng công việc. Thứ nhất, làm sao để chỉnh sửa và phát lại dùng chung một vùng vẽ mà không vô tình làm mất hiệu lực hoặc làm sai một lần chạy? Thứ hai, làm sao để các chủ đề khác nhau dùng chung bố cục nhưng vẫn giữ được tương tác riêng? Thứ ba, làm sao để một bước vẫn dễ hiểu khi nó thay đổi trạng thái bên trong nhưng tạo ra rất ít chuyển động nhìn thấy được? Các mục 4.2–4.8 mô tả giải pháp đã triển khai.

3 Quá trình phát triển và phân công

3.1 Từ các nguyên mẫu độc lập đến một ứng dụng

Giai đoạn đầu, bắt đầu từ trước học kỳ, khám phá hành vi tương tác trong các kho mã độc lập. Nguyên mẫu đồ thị đầu tiên (Graph Visualizer V1) có commit đầu tiên được ghi nhận vào ngày 28/7/2026; nguyên mẫu đồ thị thứ hai (Graph Visualizer V2) tiếp nối trong tháng 8 với việc tách trạng thái dữ liệu, bố cục và phát lại rõ ràng hơn. Các nguyên mẫu sắp xếp và cây khám phá những cách biểu diễn và mô hình thao tác khác. Công việc này tạo ra nền tảng chức năng trước khi tích hợp với website chung.

Trong tháng 9 và tháng 10, tôi điều chỉnh các bộ trực quan hóa theo hệ thống thiết kế của Kerk. Wireframe trang trực quan hóa của Kerk phác thảo chín bố cục ứng viên,

đánh số V1–V9; các số hiệu bố cục này không liên quan đến các nguyên mẫu đồ thị nêu trên. Sau một khảo sát người dùng do Kerk tổ chức, Kerk và tôi chọn hai bố cục trong số đó là V2 (các thanh bên tách biệt, split rails) và V7 (các bảng dạng dock), và tôi xây dựng các bộ trực quan hóa đồ thị, cây và sắp xếp trên cả hai bố cục với chức năng tương đương (20–27/9). Trong tuần 9, Kerk và tôi thảo luận một đề xuất kết hợp V2 và V7 thành bố cục thứ mười, V10, giữ lại các điểm mạnh được lựa chọn của mỗi phương án. Tôi triển khai V10, và Kerk đóng góp các tinh chỉnh để hoàn thiện phiên bản sau đó được dùng trong thử nghiệm. Khảo sát đóng vai trò đầu vào định hướng; tôi không lưu giữ dữ liệu câu trả lời của khảo sát, nên báo cáo không dùng nó làm bằng chứng về sở thích của người dùng.

Ngày 18/10, Kerk hợp nhất các bản sao website và bộ trực quan hóa thành một ứng dụng, đồng thời chuyển mã chung vào một lớp dùng chung. Sau đó, Kerk và tôi tổ chức một đợt thử nghiệm người dùng mang tính định hướng riêng ở tuần 10 (Mục 6.1). Công việc tiếp theo mở rộng thuật toán, bổ sung floating text và sửa các lỗi phát hiện khi rà soát. Bảng 1 tóm tắt các giai đoạn này.

Bảng 1: Các giai đoạn phát triển chính.

Thời gian	Kết quả chính
Tháng 7–8	Nguyên mẫu đồ thị đầu tiên (commit đầu tiên được ghi nhận ngày 28/7, trước học kỳ); nguyên mẫu đồ thị thứ hai với trạng thái dữ liệu, bố cục và phát lại được tách riêng; nguyên mẫu sắp xếp độc lập.
Tháng 9	Các không gian làm việc cho cây; điều chỉnh theo hệ thống thiết kế của Kerk; xây dựng bố cục V2 và V7 (20–27/9); thử nghiệm riêng về ranh giới tích hợp engine.
29/9–17/10	DFS đệ quy, Bellman–Ford và Prim; lazy segment tree và xóa trong AVL; cải tiến phát lại; thảo luận V10 (tuần 9) và triển khai.
18–31/10	Hợp nhất một ứng dụng (Kerk); thao tác LCA và heap; thử nghiệm người dùng tuần 10; sắp xếp tô-pô, kiểm tra hai phía, đỉnh khớp và cạnh cầu; floating text; sửa lỗi sau rà soát.
1–4/11	Ghi nhận kiểm chứng (1/11) và chuẩn bị báo cáo.

3.2 Đóng góp cá nhân và sự phối hợp

Các đóng góp chính của tôi là nền tảng bộ trực quan hóa độc lập, việc tích hợp chúng, mô hình bố cục cốt lõi của V10, các phần mở rộng thuật toán và thao tác, cùng lớp floating text dùng chung, kèm theo phần lớn kiểm thử tự động và hồ sơ quyết định thiết kế cho các phần này. Trong 48 lựa chọn thiết kế được đánh số của đề xuất V10, 26 lựa chọn đầu tiên là quyết định của tôi. Các lựa chọn 27–48 (18–22/10) và quyết định ngày 18/10 chỉ giữ lại V10 do Kerk và tôi cùng đưa ra, và do Kerk triển khai. Kerk cũng phụ trách hệ thống thiết kế ban đầu, nền tảng engine, hạ tầng và việc hợp nhất ứng dụng. Các trách nhiệm này giao nhau ở ranh giới tích hợp và không được suy ra chỉ từ số lượng commit [8].

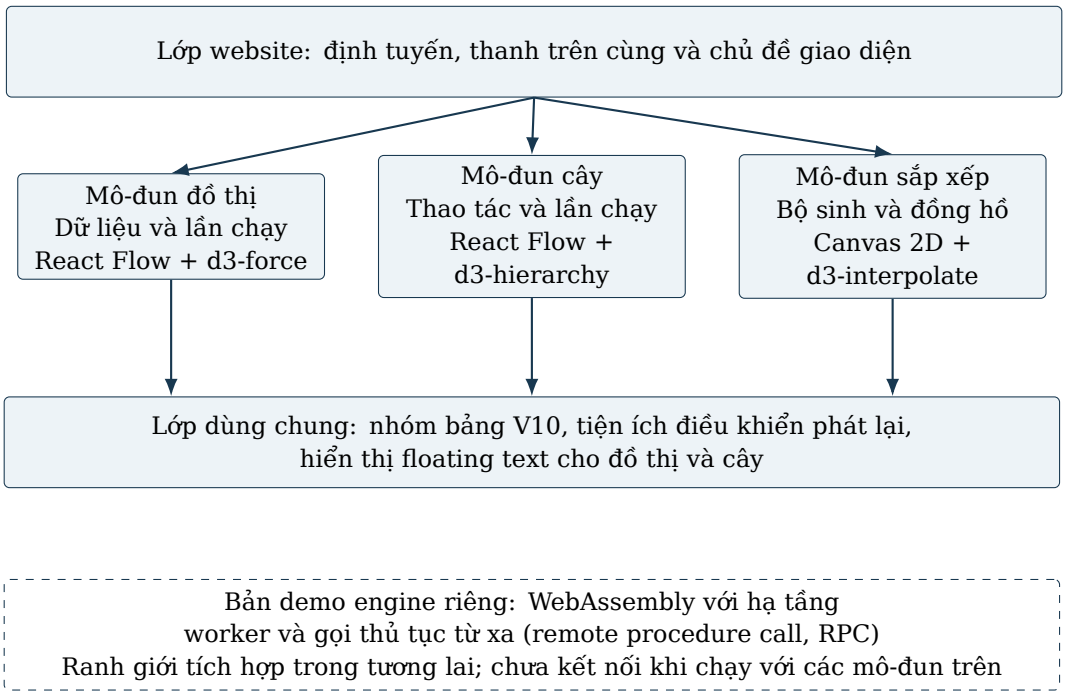
Tôi sử dụng Claude và Codex để hỗ trợ triển khai, gỡ lỗi, sinh kiểm thử và rà soát theo yêu cầu và lựa chọn thiết kế của mình. Quá trình chấp nhận thay đổi của tôi bao gồm đọc các thay đổi, chạy ứng dụng và sử dụng kết quả kiểm thử hoặc tích hợp liên tục

(CI). Trong các phiên rà soát cuối tháng 10, Codex xác định các vấn đề, Claude tái hiện chúng và tôi lựa chọn các sửa chữa cần thực hiện. AI cũng hỗ trợ tổ chức bằng chứng và soạn báo cáo này. Sự hỗ trợ đó là một phần của quy trình phát triển; mã được sinh ra và kiểm thử đạt vẫn cần được xem xét có phê phán, và tự chúng không đủ để khẳng định tính đúng đắn.

4 Thiết kế và triển khai hệ thống

4.1 Kiến trúc ứng dụng hiện tại

Bản hiện tại là một ứng dụng trang đơn dùng React và TypeScript, được xây dựng với Vite. Lớp website quản lý định tuyến, thanh trên cùng và giao diện màu sắc. Trang mô-đun chứa bộ trực quan hóa đồ thị, cây hoặc sắp xếp. Mỗi bộ trực quan hóa tự quản lý dữ liệu, thực thi thuật toán và trạng thái phát lại; các thành phần dùng chung cung cấp bố cục và tiện ích trình bày (Hình 1) [6].



Hình 1: Ranh giới của ứng dụng hiện tại. Mỗi mô-đun tự hiển thị hình vẽ của mình; khung nét đứt phía dưới là một nguyên mẫu riêng, không phải thành phần mà ứng dụng trực quan hóa đang phụ thuộc vào.

Cách tổ chức này không giao logic thuật toán cho khung giao diện chung. Ví dụ, khung giao diện quyết định vị trí bảng Status, còn thuật toán đang chạy quyết định nội dung của bảng. Trạng thái nằm trong trình duyệt; các bố cục V10 đã lưu được đánh phiên bản trong local storage.

4.2 Phân tách trạng thái và phát lại có thể tái lập

Nền tảng đồ thị tách bốn nhóm trách nhiệm (Bảng 2). Một vùng vẽ được duy trì xuyên suốt, dùng lại cho cả chỉnh sửa và thực thi. Khi bắt đầu một lần chạy, hệ thống tạo chuỗi các bước có kiểu dữ liệu xác định từ dữ liệu hiện tại; khi đi qua chuỗi đó, hệ

thống suy ra phần trình bày cần hiển thị. Điều này giảm nguy cơ duy trì riêng một đồ thị để chỉnh sửa và một đồ thị để chạy hoạt ảnh, rồi hai bản dần không còn khớp nhau.

Bảng 2: Phân tách trạng thái trong bộ trực quan hóa đồ thị.

Thành phần	Trách nhiệm	Hệ quả minh họa
Dữ liệu đồ thị	Đỉnh, cạnh, trọng số, định danh ổn định và phiên bản cấu trúc đồ thị.	Thay đổi dữ liệu có ý nghĩa thuật toán làm mất hiệu lực lần chạy trước.
Bố cục đồ thị	Vị trí và các đỉnh được ghim.	Kéo một đỉnh thay đổi hình vẽ nhưng không thay đổi đầu vào thuật toán.
Phiên chạy	Chuỗi bước, chỉ số hiện tại, định danh lần chạy và trạng thái phát lại.	Sự kiện hẹn giờ cũ không thể đẩy một lần chạy mới sang bước tiếp theo.
Trình bày	Tô sáng, nhãn thông tin và kiểu hiển thị đỉnh/cạnh được suy ra.	Lùi lại khôi phục phần trình bày của bước được chọn.

Ví dụ, di chuyển đỉnh B không nên làm thay đổi phép tính đường đi ngắn nhất; thay đổi trọng số cạnh thì có. Tách dữ liệu vị trí khỏi đầu vào mang ý nghĩa thuật toán làm rõ sự khác biệt đó. Định danh lần chạy cũng bảo vệ hệ thống trước các sự kiện phát lại bất đồng bộ đến sau khi người dùng dừng hoặc thay thế một lần chạy.

Các thuật toán đồ thị và cây tạo đầy đủ chuỗi bước ngay khi bắt đầu. Phiên chạy sau đó chuyển giữa các trạng thái chưa chạy, tạm dừng, đang phát và đã kết thúc. Mỗi bước được chọn cung cấp thông tin nhất quán cho hình vẽ, dòng mã giả được tô sáng và văn bản trạng thái. Cách tạo trước toàn bộ này giúp việc lùi bước và nhảy đến một bước đơn giản hơn, nhưng làm tăng thời gian khởi tạo và mức sử dụng bộ nhớ đối với lần chạy dài. Các chi phí này chưa được đo bằng benchmark.

Các thao tác trên cây phân biệt thêm giữa bản xem trước và dữ liệu đã được áp dụng chính thức. Người học có thể khám phá thao tác chèn hoặc xóa qua các trạng thái trung gian trước khi áp dụng kết quả cuối. Nhờ đó, họ có thể quan sát thao tác mà không coi từng khung hình là một thay đổi vĩnh viễn.

4.3 Sắp xếp sử dụng mô hình thực thi khác

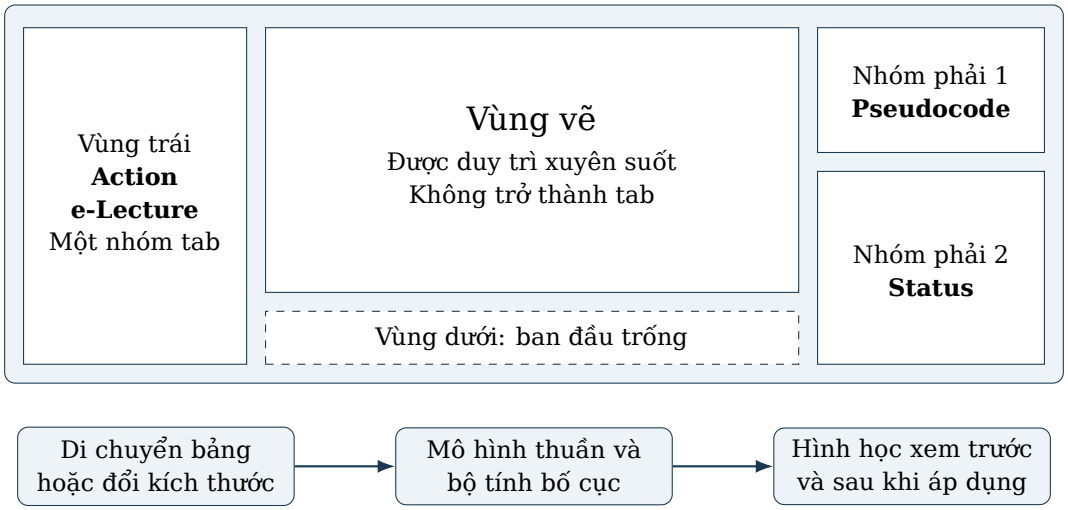
Sắp xếp sinh thao tác thông qua các bộ sinh (generator). Một đồng hồ chung điều khiển một, hai hoặc bốn ô, mỗi ô có luồng thao tác riêng. Bản triển khai giữ lịch sử đã thực hiện, đệm tối đa 2.048 thao tác tương lai và chuẩn bị trước các luồng đến ngưỡng tổng cộng 60.000 thao tác, sau đó tiếp tục sinh theo nhu cầu. Khi lùi, trạng thái được dựng lại từ các thao tác đã lưu; hệ thống không dựa vào các mốc trạng thái lưu sẵn (checkpoint).

Cách này không yêu cầu mọi thuật toán sắp xếp phải sinh xong toàn bộ thao tác trước khi người dùng bắt đầu tương tác. Tuy nhiên, lịch sử được giữ lại không bao giờ được cắt bớt (chỉ giới hạn 240 giá trị đầu vào chặn kích thước của nó), và mỗi lần lùi sẽ thực hiện lại các thao tác từ đầu đến vị trí đích, nên chi phí tăng theo độ muộn của vị

trí đích. Cả hai đều là giới hạn cần được đo lường sau này. Chỉ số thao tác chung giúp quan sát đồng bộ, nhưng thao tác của các thuật toán khác nhau không phải những đơn vị chi phí tính toán tương đương.

4.4 V10: bố cục nhóm bảng dùng chung

Khung V10 tổ chức thông tin hỗ trợ xung quanh một vùng vẽ trung tâm cố định. Vùng vẽ không thể trở thành một tab. Các vùng trái, phải và dưới chứa nhóm tab; người dùng có thể đưa một bảng vào nhóm, tách nó sang nhóm khác hoặc đặt nó vào một vùng trống. Bố cục mặc định đặt Action và e-Lecture ở bên trái, Pseudocode và Status ở bên phải (Hình 2).



Hình 2: Bố cục V10 và phép tính hình học dùng chung. Vùng dưới được vẽ như một vùng đích; khi trống, nó không giữ sẵn chiều cao như trong sơ đồ.

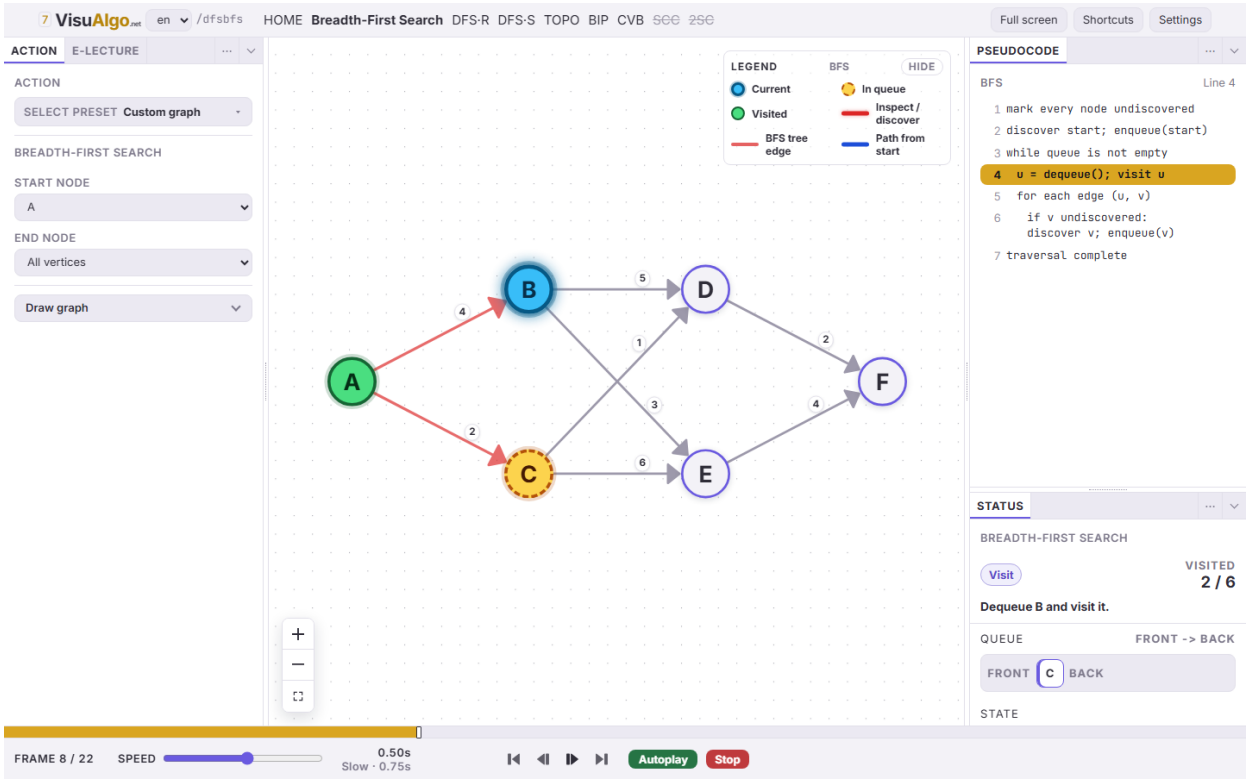
Quyết định kỹ thuật chính là biểu diễn thay đổi bố cục trong một mô hình thuần, thay vì đặt toàn bộ quy tắc vào các hàm xử lý sự kiện con trỏ. Mô hình bảo toàn quan hệ bảng thuộc nhóm nào, loại bỏ nhóm rỗng và đảm bảo tab đang được chọn của mỗi nhóm thực sự thuộc nhóm đó. Mỗi bảng bắt buộc xuất hiện đúng một lần, bảng tùy chọn xuất hiện tối đa một lần, và nhóm không lồng vào nhau.

Một bộ tính bố cục dùng chung xử lý kích thước, tính hợp lệ của đích thả và hình xem trước khi kéo. Thao tác di chuyển dự kiến trước hết được biểu diễn trong mô hình, rồi chuyển thành hình học bố cục; do đó bản xem trước và kết quả sau khi áp dụng tuân theo cùng các ràng buộc. Các nhóm đang mở rộng giữ tối thiểu 240 pixel chiều rộng và 120 pixel chiều cao. Mỗi mô-đun tự đặt ngưỡng tối thiểu cho vùng vẽ (360×320 pixel với đồ thị; 320×200 hoặc 320×240 pixel với các không gian làm việc cây), và bộ tính bố cục giữ ngưỡng này khi vùng hiển thị đủ chỗ.

Bố cục lưu lại có phiên bản; trạng thái không hợp lệ sẽ được sửa hoặc thay bằng mặc định. Kiểm thử tự động và kiểm tra thủ công được dùng để kiểm chứng hành vi bố cục: kiểm thử mô hình chạy 4.000 thao tác ngẫu nhiên có seed cố định trên nhiều kích thước vùng hiển thị, kèm nhiều lần chuyển trạng thái sang dạng lưu trữ rồi đọc lại; kiểm thử trình duyệt di chuyển tab qua menu tab và kéo các thanh chia (splitter); còn các checklist thủ công bao quát thao tác kéo-tách bằng con trỏ, phản hồi khi thả

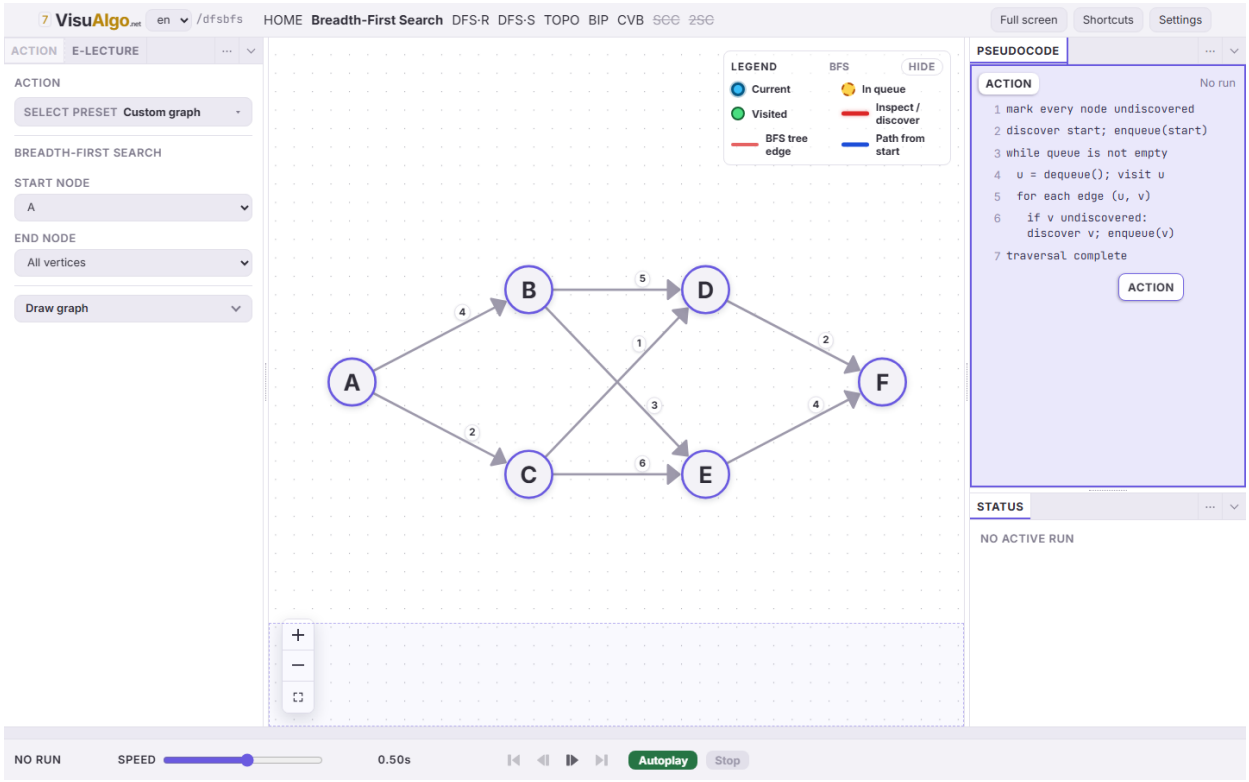
và đổi kích thước.

4.5 Khung giao diện chung trong sử dụng



Hình 3: BFS ở khung hình 8 trên 22 trong bố cục V10 mặc định. Hình vẽ, mã giả, trạng thái và bộ điều khiển phát lại ở chân trang cùng phản ánh một bước đang được chọn.

Hình 3 và 4 cho thấy vì sao bố cục là một phần của tương tác với thuật toán. Người học có thể cần nhiều chỗ hơn cho mã giả khi theo dõi một lần chạy, rồi cần nhiều chỗ hơn để chỉnh sửa đầu vào. Khung giao diện chung cho phép điều chỉnh mà không thay đổi trạng thái thuật toán bên dưới. Thiết kế này được dùng cho cả ba nhóm bộ trực quan hóa.

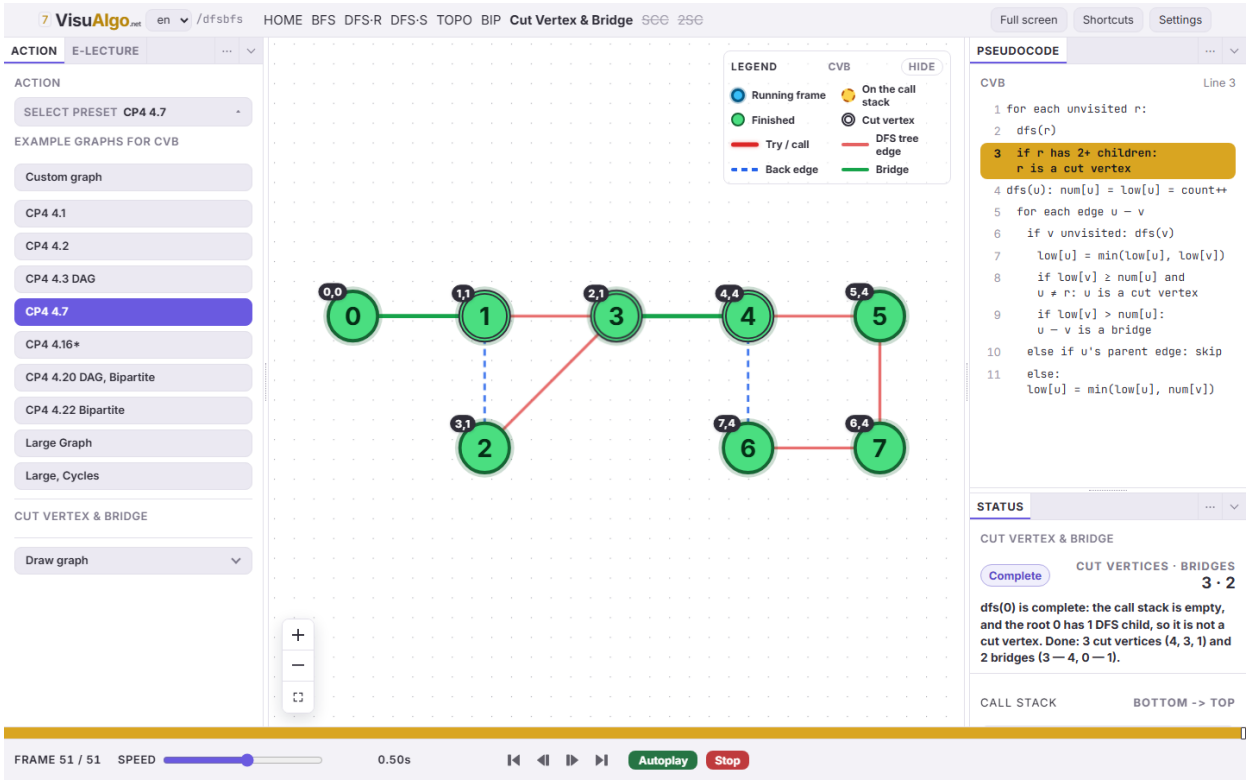


Hình 4: Kéo tab Action vào nhóm Pseudocode sẽ hiển thị vùng thả dự kiến. Ảnh chụp ghi lại bản xem trước, chưa phải thao tác thả đã hoàn thành.

4.6 Mở rộng đồ thị và kiểm chứng thuật toán

Mô-đun đồ thị hiện có 11 thuật toán, tính riêng DFS (depth-first search, tìm kiếm theo chiều sâu) đệ quy và DFS dùng ngăn xếp. Bên cạnh BFS (breadth-first search, tìm kiếm theo chiều rộng), DFS dùng ngăn xếp, Dijkstra, Kruskal và Edmonds–Karp, ứng dụng hỗ trợ DFS đệ quy, Bellman–Ford, Prim, sắp xếp tô-pô, kiểm tra đồ thị hai phía và tìm đỉnh khớp/cạnh cầu. Sắp xếp tô-pô có phương pháp DFS và Kahn; kiểm tra hai phía có phương pháp DFS và BFS. BFS, Dijkstra và Bellman–Ford cũng nhận một đỉnh kết thúc (End node) tùy chọn; khi đó khung hình cuối vẽ đường đi ngắn nhất đến đỉnh này.

Những bổ sung này mở rộng cả phần tính toán và giải thích. DFS đệ quy làm rõ cấu trúc thực thi khác với cách duyệt dùng ngăn xếp. Sắp xếp tô-pô và kiểm tra hai phía cung cấp thông tin làm chứng khi đầu vào vi phạm điều kiện, thay vì chỉ hiển thị kết quả thành công hoặc thất bại. Bellman–Ford và Prim được lên kế hoạch, xây dựng cùng Codex theo yêu cầu và sự rà soát của tôi.



Hình 5: Khung hình cuối của tìm đỉnh khớp và cạnh cầu trên đồ thị ví dụ của VisuAlgo có nhãn CP4 4.7: kết quả báo các đỉnh khớp 1, 3, 4 và các cạnh cầu 0–1, 3–4.

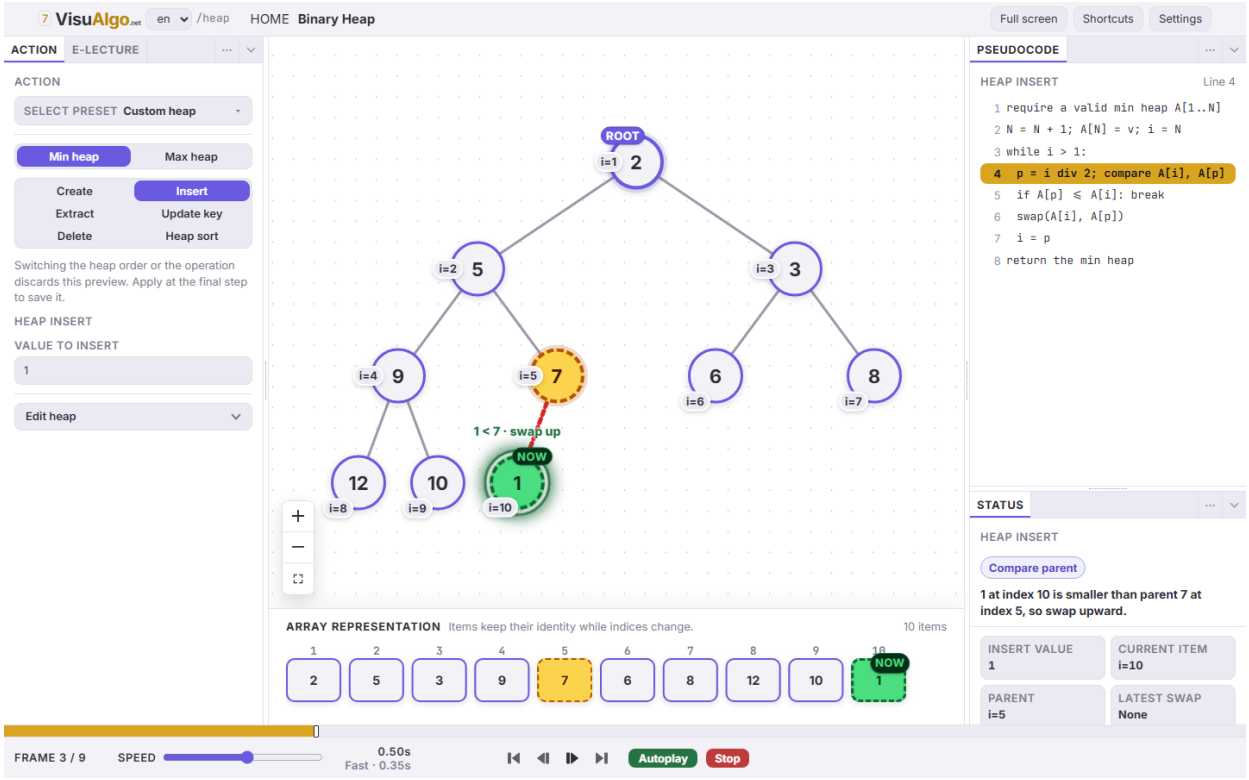
Một vấn đề đúng đắn tiêu biểu: cạnh song song. DFS low-link phải bỏ qua *cạnh* dẫn đến cha, không phải mọi cạnh dẫn đến *đỉnh* cha. Nếu hai đỉnh được nối bởi hai cạnh song song, xóa một trong hai cạnh vẫn giữ chúng liên thông. Bỏ qua cả hai cạnh trong DFS có thể khiến hệ thống đánh dấu sai một cạnh là cầu. Định danh cạnh ổn định giúp thể hiện chính xác quy tắc cần dùng; Hình 5 cho thấy kết quả cuối cùng của bộ phát hiện trên một đồ thị ví dụ của VisuAlgo.

Để kiểm chứng, các kiểm thử đối chiếu kết quả với các bộ tính kết quả tham chiếu (oracle) được xây dựng theo cách độc lập về thuật toán. Đỉnh khớp và cạnh cầu được kiểm tra bằng cách xóa từng đỉnh hoặc cạnh rồi tính lại thành phần liên thông. Bellman–Ford được đối chiếu với kết quả dựa trên Floyd–Warshall, gồm cả khả năng đi đến hoặc đi qua chu trình âm. Prim được so với các phương án rừng được liệt kê vét cạn trên đầu vào nhỏ. Các oracle khác về thuật toán so với bản triển khai đang kiểm thử; điều này giảm các giả định chung, nhưng không phải chứng minh hình thức.

Các trường hợp ngẫu nhiên được ghi nhận gồm 700 đa đồ thị (multigraph) ngẫu nhiên cho đỉnh khớp và cạnh cầu (400 đồ thị đối chiếu với oracle xóa đỉnh/cạnh và 300 đồ thị kiểm tra quy tắc gốc DFS), 140 đồ thị có trọng số âm hoặc dương cho Bellman–Ford và 20 đồ thị nhỏ cho Prim, thử từ mọi đỉnh bắt đầu. Các phương pháp sắp xếp tô-pô và kiểm tra hai phía có thêm kiểm thử ngẫu nhiên riêng. Seed cố định giúp tái hiện lỗi.

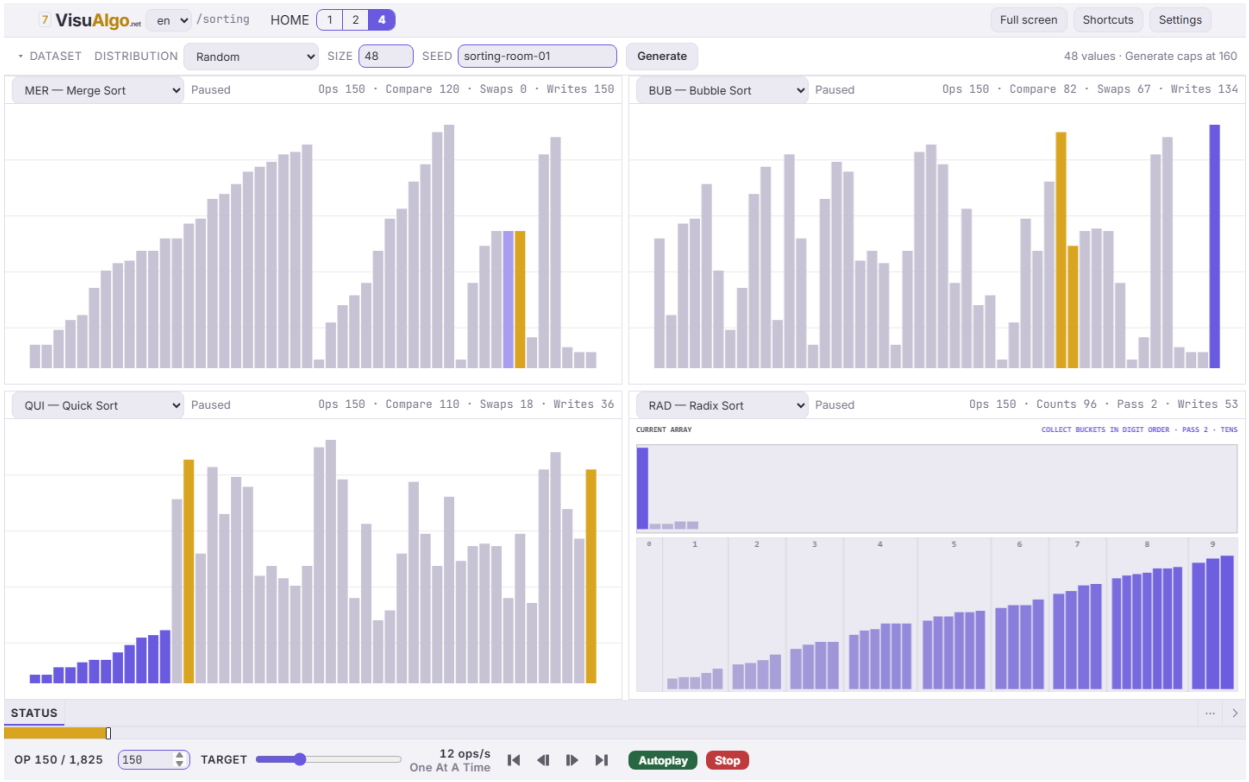
4.7 Thao tác trên cây và so sánh sắp xếp

Bộ trực quan hóa cây có bảy không gian làm việc: LCA (lowest common ancestor, tổ tiên chung gần nhất), BST (binary search tree, cây tìm kiếm nhị phân), cây AVL, binary heap, trie, segment tree và lazy segment tree. Công việc bao gồm truy vấn và duyệt BST/AVL, xóa trong AVL, các thao tác heap và hành vi cộng đoạn/truy vấn tổng đoạn với lazy propagation. Heap liên kết cách biểu diễn mảng và cây; các thay đổi trung gian vẫn là bản xem trước cho đến khi được áp dụng (Hình 6). Có những không gian làm việc này không đồng nghĩa với đã có đầy đủ mọi thao tác tương ứng của VisuAlgo.



Hình 6: Chèn 1 vào heap, khung hình 3 trên 9: phần tử mới ở chỉ số 10 được so sánh với phần tử cha 7 ở chỉ số 5. Ghi chú "$1 < 7 \cdot$ swap up" nêu quyết định trước khi đổi chỗ, và dải mảng đánh dấu cùng hai phần tử đó.

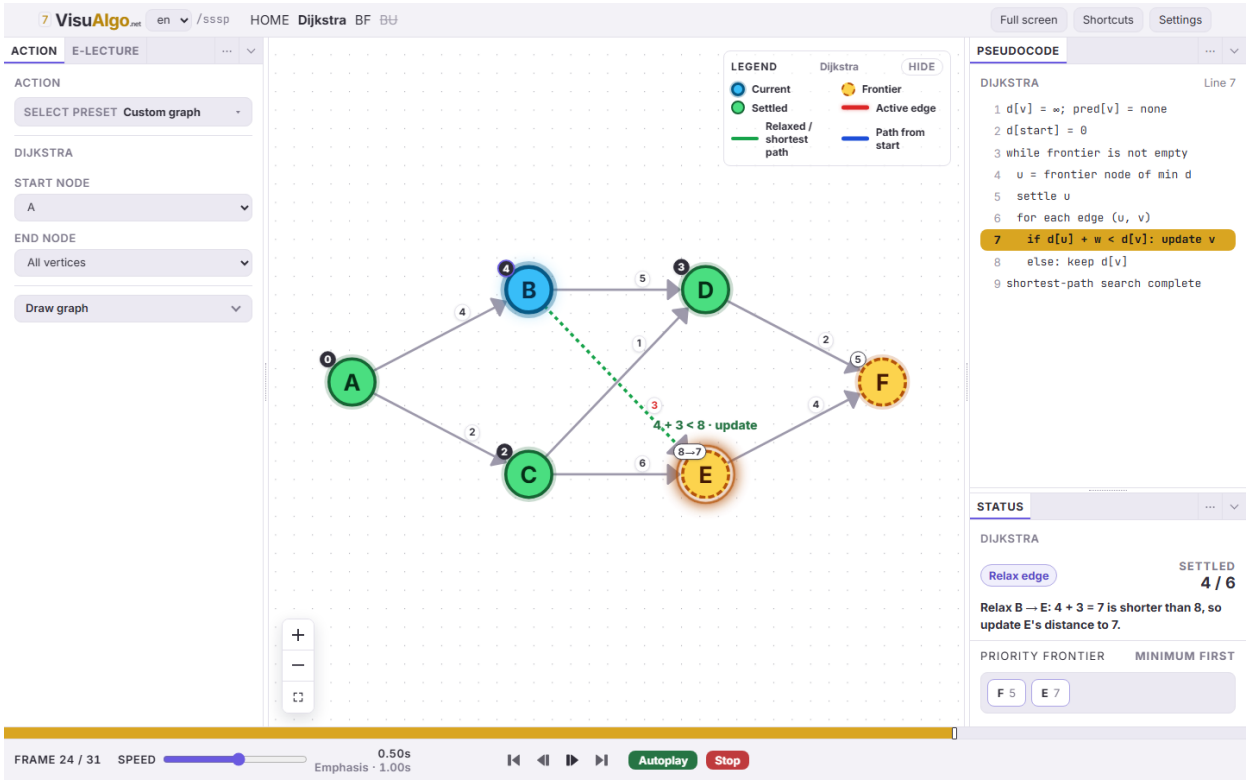
Phần sắp xếp có tám thuật toán trong chế độ Study một ô và Compare hai hoặc bốn ô. Các ô dùng chung dữ liệu đầu vào và đồng hồ, cho phép quan sát các chuỗi thao tác khác nhau trên cùng dữ liệu (Hình 7). Hệ thống ngăn chọn trùng thuật toán và quicksort ngẫu nhiên sử dụng seed có thể tái lập.



Hình 7: So sánh bốn ô Merge, Bubble, Quick và Radix sort trên cùng đầu vào 48 phần tử. Chỉ số thao tác chung là tọa độ phát lại, không phải phép đo thời gian chạy tương đối.

4.8 Floating text: lời giải thích ngay trên vùng vẽ

Một bước có thể có ý nghĩa ngay cả khi hình vẽ hầu như không thay đổi. Ví dụ, Dijkstra có thể từ chối một lần cập nhật khoảng cách (relaxation) vì khoảng cách ứng viên không ngắn hơn khoảng cách hiện có. Nếu lý do chỉ xuất hiện trong bảng Status riêng, người học phải liên tục rời mắt khỏi cạnh liên quan. Sau đợt thử nghiệm tuần 10, tôi bổ sung floating text: các ghi chú ngắn cho từng bước, đặt trên vùng vẽ ngay tại đỉnh liên quan (với Dijkstra là đỉnh mà cạnh đang xét đi tới). Đây là một trong vài cải tiến UI/UX sau thử nghiệm, bên cạnh các điều chỉnh cho phần chú giải (legend) và bảng Status, nhằm giảm số lần người dùng phải nhìn sang Status khi theo dõi một lần chạy và giữ sự chú ý của họ trên vùng vẽ. Status vẫn giữ nội dung chi tiết để đọc lại; giảm chuyển hướng chú ý là mục tiêu thiết kế, chưa phải kết quả đã đo lường.



Hình 8: Dijkstra xét cập nhật từ B đến E: ghi chú giải thích $4 + 3 < 8$, nên khoảng cách của E đổi từ 8 thành 7.

Hình 8 minh họa một lần cập nhật được chấp nhận. Mỗi lần xét cập nhật ghi một ghi chú trên đỉnh đích của nó: màu xanh lá khi khoảng cách thay đổi, như ở đây, và màu đỏ khi khoảng cách được giữ nguyên, chẳng hạn "already settled". Bản triển khai có mô tả ghi chú, phép tính vị trí, logic thời gian và lớp hiển thị dùng chung. Thuật toán đồ thị và cây cung cấp nội dung có ý nghĩa; lớp dùng chung điều khiển cách hiển thị. Hiện đã áp dụng cho Dijkstra, LCA, BST, AVL và Heap. Sắp xếp, Trie, các cây đoạn và các thuật toán đồ thị khác chưa sử dụng lớp này.

Ghi chú được hiển thị bên ngoài phép biến đổi thu phóng (zoom transform) của vùng vẽ đồ thị hoặc cây, nên cỡ chữ giữ nguyên trên màn hình trong khi mỗi ghi chú đi theo đỉnh của nó. Việc bố trí kiểm tra những vật cản gần đó, gồm nhãn thông tin trên đỉnh và nhãn trọng số cạnh. Hệ thống thử các vị trí ưu tiên trước, rồi các phương án thay thế; nếu không có vị trí trống, nó chọn phương án ít chồng lấn nhất. Vì vậy, đồ thị dày vẫn có thể gây chồng lấn.

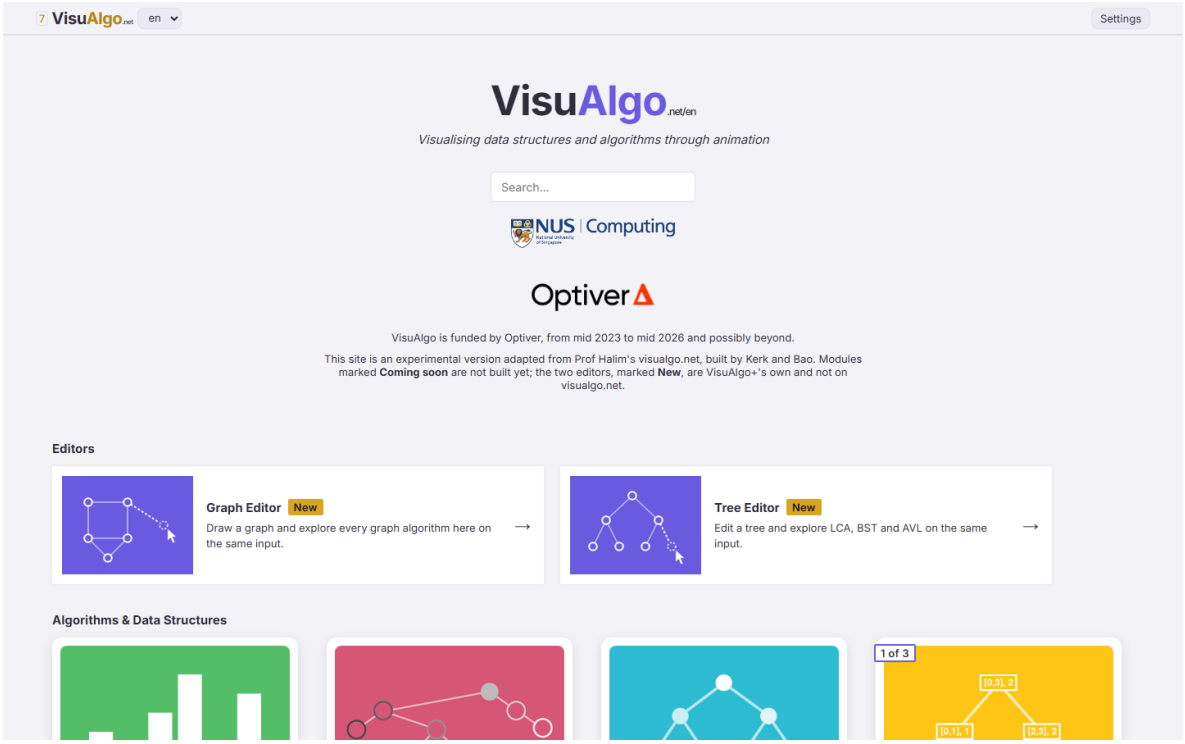
Thời gian hiển thị thích ứng với phát lại. Ghi chú chỉ xuất hiện khi một bước được đến bằng cách tiến thêm một bước (Next hoặc tự động phát). Sau Next, ghi chú tồn tại 1.600 mili giây; khi tự động phát, thời lượng được rút xuống còn 90% khoảng thời gian mỗi bước, và ghi chú bị bỏ qua khi thời lượng còn lại dưới 200 mili giây, nhờ vậy phát lại nhanh không làm tích tụ văn bản không thể đọc.

Vì chỉ xem trực quan hóa hiếm khi cải thiện việc học [3], các ghi chú sẽ được đánh giá qua các nhiệm vụ chủ động (Bảng 4). Lớp ghi chú hiện được ẩn khỏi các API hỗ trợ tiếp cận, và ứng dụng chưa được đánh giá khả năng tiếp cận một cách có hệ thống. Công việc sau này cần kiểm tra liệu phần văn bản tương đương được giữ lại có đủ hay không, và người dùng có thể hiểu các thao tác bị từ chối, chuyển đổi nhanh và

trường hợp dày đặc mà không chỉ dựa vào màu sắc hoặc chuyển động hay không.

5 Kết quả hiện tại và kiểm chứng

Bản hiện tại cung cấp một website với các mô-đun đồ thị, cây và sắp xếp chạy được (Hình 9). Trong 30 thẻ danh mục, 12 thẻ mở được mô-đun và 18 thẻ được đánh dấu sắp có (coming soon). Trong 26 thẻ được chép lại từ danh sách mô-đun của visualgo.net, tám thẻ mở được: bốn thẻ có đủ mọi thuật toán mà mô-đun gốc liệt kê (Sorting, BST/AVL, Binary Heap và Minimum Spanning Tree) và bốn thẻ có một phần (Segment Tree, Graph Traversal, Single-Source Shortest Paths và Max Flow). Bốn thẻ mở được còn lại là Trie và LCA, vốn là các trang riêng trên visualgo.net, cùng Graph Editor và Tree Editor của chính dự án này. Các nhãn này đếm thuật toán, không đếm thao tác, và không phải kết quả kiểm chứng giáo dục.



Hình 9: Phần đầu trang chủ của ứng dụng tích hợp. Dòng ghi công cho biết website được chuyển thể từ visualgo.net; danh mục mô-đun, gồm cả các thẻ sắp có (coming soon), tiếp tục ở phía dưới.

Bảng 3: Các kiểm tra được ghi nhận ngày 1/11/2026 tại commit ứng dụng `662e7b3` [7].

Kiểm tra	Kết quả	Ý nghĩa
Kiểm thử đơn vị đồ thị	654 đạt	Thuật toán, trạng thái và hành vi đồ thị.
Kiểm thử đơn vị cây	1.881 đạt	Thao tác, trạng thái và trình bày.
Kiểm thử đơn vị sắp xếp	571 đạt	Luồng thao tác và phát lại.
Kiểm thử lớp chung / website	143 / 24 đạt	Thành phần dùng chung và logic website.
Tổng kiểm thử đơn vị	**3.273 đạt**	Tổng bộ kiểm thử được ghi nhận.
Ca kiểm thử trình duyệt	151 đạt	14 tệp đặc tả kiểm thử trên Chromium.
Kiểm tra kiểu / build	Đạt	Kiểm tra lúc biên dịch và bản dựng production.
Lint	0 lỗi	Còn hai cảnh báo.

Môi trường được ghi nhận là Windows 11, Node 24.17.0, npm 11.13 và Playwright 1.64.1 với Chromium. Quá trình build phát cảnh báo về gói mã lớn (khoảng 705 kB trước gzip và 206 kB sau gzip). Từ ngày 4/9 đến 30/10, CI của kho mã đã chạy 266 lượt workflow, trong đó 105 lượt thuộc các workflow của bộ trực quan hóa, số còn lại thuộc CI engine của Kerk và việc triển khai website; tám lượt thất bại, và mỗi lượt đều được tiếp nối bởi một lượt chạy thành công của cùng workflow.

Kiểm thử đạt giúp tăng độ tin cậy rằng các trường hợp đã kiểm tra không bị lỗi hồi quy (Bảng 3); chúng không đo tính dễ sử dụng, khả năng tiếp cận, khả năng hoạt động trên các trình duyệt khác hay kết quả học tập. Kiểm thử tự động và kiểm tra thủ công được dùng để kiểm chứng hành vi giao diện (Mục 4.4). Thử nghiệm tuần 10 cung cấp phản hồi định hướng về tính dễ sử dụng (Mục 6.1).

6 Đánh giá người dùng, giới hạn và bước tiếp theo

6.1 Thử nghiệm người dùng ở tuần 10

Trong tuần 10, Kerk và tôi mời một số người dùng, đều là sinh viên NUS, tự sử dụng VisuAlgo+ V10 trong 15–20 phút. Người tham gia có mức độ quen thuộc với VisuAlgo.net khác nhau, từ đã sử dụng nhiều lần đến ít sử dụng; kinh nghiệm thuật toán cũng trải rộng từ người có kinh nghiệm lập trình thi đấu đến người mới bắt đầu. Chúng tôi quay video từng phiên, cho người tham gia làm một bài test ngắn sau khi sử dụng và thu thập phản hồi. Kerk đã xin phép người tham gia từ trước về việc quay và sử dụng video, và tất cả đều đồng ý; video chỉ ghi màn hình và thao tác tay, không ghi khuôn mặt.

Các phiên thử nghiệm mang lại phản hồi tích cực cùng những điểm cần cải thiện. Xem lại video giúp chúng tôi nhận ra những tương tác hỗ trợ người dùng làm quen với giao diện mới và những chỗ cần điều chỉnh thêm. Từ đó có vài cải tiến UI/UX: floating text, giúp đưa lời giải thích đến gần hình vẽ (Mục 4.8), cùng các điều chỉnh

cho phần chú giải (legend) và bảng Status. Đợt thử nghiệm này tách biệt với khảo sát bố cục trước đó của Kerk.

6.2 Giới hạn bằng chứng và triển khai

Số người tham gia, đề test, điểm số và các quan sát chi tiết từ đợt thử nghiệm chưa được tổng hợp, nên báo cáo này không trích dẫn. Đợt thử nghiệm là đầu vào định hướng thiết kế; chỉ một bài test sau sử dụng chưa chứng minh mức tiến bộ học tập hay tính dễ sử dụng so với hệ thống khác, và tác động của floating text chưa được đo lường. Benchmark hiệu năng và tỷ lệ bao phủ mã cũng chưa có.

Một số giới hạn triển khai có ý nghĩa thực tế. Đồ thị và cây tạo trước toàn bộ chuỗi bước. Sắp xếp không bao giờ cắt bớt lịch sử và thực hiện lại thao tác khi nhảy vị trí. Floating text chỉ có ở một số thuật toán và có thể chồng lấn trong hình vẽ dày. Kiểm chứng trình duyệt tập trung vào Chromium trên máy tính; hành vi trên điện thoại, tương thích nhiều trình duyệt và khả năng tiếp cận vẫn chưa hoàn thiện. Các checklist thủ công của người phát triển (20, 10 và 26 mục) đã được chạy, nhưng các lần chạy không được ghi lại; chúng tách biệt với thử nghiệm tuần 10.

Engine riêng cũng là một ranh giới phạm vi. Thử nghiệm chưa được hợp nhất của tôi trên engine của Kerk khám phá định danh cạnh ổn định và một client trình duyệt tái sử dụng được để quản lý vòng đời worker/RPC. Nó giúp xác định yêu cầu tích hợp, nhưng không nằm trên đường thực thi của bộ trực quan hóa đã triển khai.

6.3 Đề xuất đánh giá tiếp nối

Trước hết, cần tổng hợp video, câu trả lời bài test và phản hồi tuần 10 thành các quan sát đã ẩn danh, liên kết với những thay đổi thiết kế (Bảng 4). Sau đó thực hiện đánh giá tiếp nối có cấu trúc hơn, dự kiến gồm 6–10 sinh viên đã biết cấu trúc dữ liệu nhập môn. Kế hoạch người tham gia, quy trình xin đồng ý (consent) và mọi yêu cầu xét duyệt đạo đức nghiên cứu của NUS sẽ được xác nhận với giảng viên hướng dẫn trước khi tuyển người. Một nghiên cứu nhỏ có thể tìm ra vấn đề tương tác, nhưng không đủ để khẳng định hiệu quả học tập nói chung.

Bảng 4: Nhiệm vụ đề xuất cho đợt tiếp nối; không phải đề test đã dùng ở tuần 10.

Nhiệm vụ	Bằng chứng cần thu thập
Giải thích cập nhật Dijkstra	Giải thích đúng một lần cập nhật được chấp nhận và một lần bị từ chối; cách dùng ghi chú, mã giả và Status.
Theo dõi chèn vào heap	Dự đoán đúng lần đổi chỗ tiếp theo và hiểu khác biệt giữa xem trước và Apply.
Sắp xếp lại không gian làm việc	Di chuyển/tách tab thành công, khôi phục sau bố cục không mong muốn, lỗi và mức hỗ trợ cần thiết.
So sánh hai lần sắp xếp	Giải thích khác biệt thao tác mà không coi chỉ số thao tác là thời gian chạy đã trôi qua.

Ghi nhận mức hoàn thành nhiệm vụ, thời gian, lỗi, yêu cầu trợ giúp và nhận xét ngắn của người tham gia. Nếu có thể so sánh với VisuAlgo gốc, dùng nhiệm vụ và đầu vào

tương ứng, đồng thời cân bằng thứ tự trải nghiệm giữa các người tham gia.

6.4 Đề xuất đo lường kỹ thuật

Dùng một máy cố định và bản build production, với các đầu vào đã lưu có kích thước tăng dần. Đo thời gian chuẩn bị lần chạy, thời gian đến bước đầu tiên hiển thị, độ trễ nhảy bước, mức đều đặn của khung hình khi phát lại và lượng bộ nhớ sử dụng. Lặp mỗi điều kiện ít nhất năm lần, báo cáo trung vị cùng mức phân tán và môi trường thực hiện. Những phép đo này sẽ định hướng việc có cần checkpoint, giới hạn lịch sử hay thay đổi ranh giới thực thi hay không.

7 Kế hoạch cho phần còn lại của dự án

Kế hoạch dưới đây (Bảng 5) được đề xuất để trao đổi với giảng viên hướng dẫn và Kerk. Các mốc cố định của SoC là buổi thuyết trình CA trong Reading Week (14–20/11/2026), hạn nộp báo cáo cuối lúc 17:00 thứ Tư, ngày 7/4/2027, và buổi thuyết trình cuối ngày 19–23/4/2027 (dự kiến); các khoảng thời gian khác là đề xuất.

Bảng 5: Các mốc và bằng chứng hoàn thành được đề xuất.

Thời gian	Công việc chính	Bằng chứng hoàn thành
11–12/2026	Thuyết trình CA; tổng hợp phát hiện tuần 10; ổn định bản hiện tại; xác định nhiệm vụ tiếp nối; cập nhật thường xuyên các kiểm tra UI tự động và thủ công.	Mốc cơ sở có thể tái lập, kiểm thử UI và checklist đã cập nhật, cùng quy trình đánh giá đã rà soát.
1/2027	Cùng Kerk nghiên cứu tích hợp engine trong một phạm vi nhỏ, bắt đầu bằng thuật toán đồ thị đơn giản.	Đặc tả đầu vào và bước chạy; kết quả và ngữ nghĩa phát lại khớp bản tham chiếu TypeScript.
Tháng 2	Đánh giá tiếp nối theo nhiệm vụ và thực hiện đo lường kỹ thuật.	Quan sát đã ẩn danh, đầu vào benchmark cố định, môi trường và bảng kết quả.
Tháng 3	Xử lý phát hiện có ảnh hưởng lớn; cải thiện giải thích và khả năng tiếp cận ở các mô-đun được chọn; tổng hợp kết quả và soạn bản thảo báo cáo cuối.	Kiểm tra hồi quy trước/sau, kiểm chứng lại những nhiệm vụ đã thay đổi và bản thảo báo cáo hoàn chỉnh.
Tháng 4	Nộp báo cáo cuối trước ngày 7/4; thuyết trình và demo cuối (19–23/4, dự kiến); nộp bản điện tử (e-copy) trước ngày 10/5.	Phiên bản phát hành có thể tái lập, báo cáo đánh giá cuối và tài liệu thuyết trình.

7.1 Chiến lược tích hợp và rủi ro

Tích hợp engine nên bắt đầu bằng một luồng hoàn chỉnh trong phạm vi nhỏ: đầu vào đồ thị, engine thực thi, chuyển đổi sang các bước có kiểu xác định, hiển thị và phát lại tiến/lùi. Mốc đầu tiên là sự thống nhất hành vi, gồm cả định danh cạnh và hủy

thao tác, thay vì thay thế mọi thuật toán TypeScript. Nếu đặc tả engine hoặc tiến độ chưa sẵn sàng, bản triển khai hiện có vẫn là mốc cơ sở hoạt động được, và báo cáo có thể trình bày riêng nghiên cứu tích hợp.

Rủi ro kỹ thuật chính là chi phí của chuỗi bước dài, ngữ nghĩa không nhất quán giữa các mô-đun và lỗi tương tác mà kiểm thử mô hình không phát hiện được. Cần đo lường trước khi thiết kế lại phần phát lại trên quy mô lớn. Chỉ nên tạo lớp trừu tượng dùng chung khi hành vi thực sự giống nhau; chuỗi tạo trước của đồ thị/cây và bộ sinh của sắp xếp đã cho thấy điều khiển giống nhau không bắt buộc phần thực thi bên trong giống nhau.

Rủi ro dự án chính là phạm vi ngày càng mở rộng. Thêm mọi chủ đề chưa hoàn thành trong danh mục sẽ giảm thời gian đánh giá và hoàn thiện. Tôi đề xuất ưu tiên luồng làm việc đồ thị, cây và sắp xếp hiện có, một số khoảng trống trong giải thích và một thử nghiệm engine nhỏ. Các tính năng tiếp theo nên được lựa chọn cùng giảng viên hướng dẫn dựa trên giá trị giáo dục, phần chức năng còn thiếu và thời gian sẵn có.

7.2 Kết quả kỳ vọng

Đến đợt đánh giá cuối, kết quả hướng tới là một nền tảng trực quan hóa được kiểm thử và tài liệu hóa, có giao diện nhất quán, bao phủ các thuật toán đại diện, có bằng chứng từ nhiệm vụ người dùng và đo lường kỹ thuật, cùng mô tả rõ ràng về tiến độ tích hợp engine. Công việc học kỳ đầu tạo ra nền tảng triển khai.

Tài liệu tham khảo

[1] D3 Project. Force simulation and tree layout documentation. https://d3js.org/d3-force; https://d3js.org/d3-hierarchy/tree. Accessed 3 November 2026.

[2] Steven Halim, Zi Chun Koh, Victor Bo Huai Loh, and Felix Halim. Learning algorithms with unified and interactive web-based visualization. *Olympiads in Informatics*, 6:53–68, 2012. https://ioi.te.lv/oi/shtm/INFOL099.shtml.

[3] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13(3):259–290, 2002. https://doi.org/10.1006/jvlc.2002.0237.

[4] OpenDSA Project. About OpenDSA. https://opendsa-server.cs.vt.edu/home/about. Accessed 4 November 2026.

[5] Clifford A. Shaffer, Matthew L. Cooper, Alexander Joel D. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education*, 10(3):9:1–9:22, 2010. https://doi.org/10.1145/1821996.1821997.

[6] Truong Van Quoc Bao. VisuAlgo+ project records: Architecture and decisions, 2026. Working project record (file 03), snapshot 67ce56c; describes application 662e7b3. Available on request.

[7] Truong Van Quoc Bao. VisuAlgo+ project records: Figures and validation, 2026. Working project record (file 04), snapshot 67ce56c; recorded tests dated 1 November 2026. Available on request.

[8] Truong Van Quoc Bao. VisuAlgo+ project records: Scope, contributions and report mapping, 2026. Working project records (files 01, 02 and 05), snapshot `67ce56c`, 3 November 2026. Available on request.

[9] xyflow. React Flow: Documentation. https://reactflow.dev/learn. Accessed 3 November 2026.

Hồ sơ dự án [6, 7, 8] là hồ sơ làm việc của tác giả, không phải công trình nghiên cứu; có thể cung cấp khi có yêu cầu. Ảnh chụp thể hiện commit ứng dụng `662e7b3`; sơ đồ được vẽ cho báo cáo này. VisuAlgo vẫn là nguồn của tài liệu giáo dục và các ví dụ gốc.

Ghi chú bản dịch. Tên công trình và thông tin thư mục được giữ nguyên để đối chiếu. Bản dịch đồng bộ với bản tiếng Anh trong cùng phiên bản.